

PHASE 5 SPECIFICATION CLARIFICATION

Mentor Clarification Incorporated

1. `UserSettings.currency` is the single source of truth for the user's active currency.
2. Currency is selected during onboarding and locked after onboarding for the MVP.
3. UI must use `userSettingsProvider` and the shared `CurrencyFormatter`.

CURRENCY STATE & FORMATTING CONTRACT

The authoritative flow is:

`UserSettings.currency`

Trimly - Project Engineering Documentation

Historical engineering record | Phases 1-4, Phase 5A, and frozen Phase 5 documentation | 14 September 2026

Project Overview

PHASE 5 VISUAL DESIGN SPECIFICATION

STATUS: FROZEN

Primary Visual Style

Trimly uses premium minimalism: polished, modern, trustworthy, premium, clean, financially serious, and easy to understand. The

Glassmorphism and 3D Restrictions

Glassmorphism is permitted only in the Optimizer's Obvious Guess to Trimly Recommendation reveal card. It is not used across Hom

Optimizer Reveal Animation

OBVIOUS GUESS -> REJECTED -> TRIMLY RECOMMENDATION -> WHY

The Obvious Guess appears muted or grayscale, transitions away with an elegant native fade, strike-through, slide, or combinatio

Secondary priorities are Home spend count-up, Optimizer analyzing checklist, standard page/card transitions, Value Check selecti

Frozen Palette

Role	Color
Deep Teal	#006064
Brushed Steel	#90A4AE
Vibrant Amber Gold	#FFB300

Brushed Steel is a flat neutral. Metallic textures and artificial brushed-metal effects are prohibited.

Screen Direction

- **HOME:** Premium minimalism; prioritize Spend, Renewal, Value, Savings, and Action with only subtle count-up motion.
- **VALUE CHECK:** Clean cards and subtle scale/state feedback with human-readable decisions.
- **SAVINGS MISSION:** Responsive but restrained target setting with calm milestone feedback and no confetti.
- **OPTIMIZER:** Strongest treatment only for Obvious Guess, transition, Trimly Recommendation, and reason.
- **ALTERNATIVES:** Clean comparison/list with the primary recommendation dominant.
- **RENEWAL RADAR:** Minimal timeline/list using hierarchy and spacing.
- **TRIALS:** Minimal warning-oriented status, post-trial price, renewal timing, and action.
- **PROFILE / SETTINGS:** Simple premium-minimal UI without broad glass or 3D effects.

Performance and Non-Goals

Avoid unnecessary BackdropFilter, simultaneous blur layers, expensive continuous animations, complex 3D transforms, unnecessary

Design Principle

Cut visual complexity everywhere except where the product's decision-making becomes visible.

Final Visual Specification Status

STATUS: FROZEN

Trimly is a Flutter application foundation for subscription review and savings workflows. The architecture is intentionally stag

Phase History

- Phase 1 - Flutter project foundation and frozen architecture scaffold
- Phase 2 - Domain data models and Hive persistence foundation

- Phase 3 - Decision Engine scoring and classification
- Phase 4 - Optimizer, baseline, and alternatives
- Currency & Internationalization Amendment v1 - approved and frozen
- Phase 5 UI/UX Specification - frozen handoff
- Phase 5A - UserSettings and currency foundation
- Phase 6A - RevenueCat foundation

Phase 1 - Flutter Foundation

Objective

Prepare a compiling Flutter project foundation without implementing product screens, integrations, or business behavior.

Implemented

- Flutter and Dart project bootstrap named Trimly.
- Riverpod application scope.
- App shell, router placeholder, theme placeholder, and frozen architecture directories.
- Placeholder boundaries for RevenueCat and notifications without integrating either service.
- Hive, Hive Flutter, Shared Preferences, Riverpod, Hive Generator, and Build Runner dependencies.
- Trimly branding in the application shell and platform metadata.

Intentionally Not Implemented

Home UI, subscription screens, Value Check, Savings Mission, Optimizer, RevenueCat, notifications, Decision Engine logic, AI/ML,

Validation

- flutter pub get: passed.
- dart analyze: passed.
- flutter test: passed.
- Chrome launch: passed.
- Windows launch was blocked by the host's missing Visual Studio C++ toolchain.

Phase 2 - Domain Models and Hive Persistence

Objective

Implement the frozen subscription data model, strongly typed enums, explicit Hive persistence metadata, and concrete repository

Implemented

- Subscription model with the specified fields only.
- Strongly typed UsageFrequency, Importance, DecisionStatus, BillingCycle, and SavingsAction enums.
- Generated Hive adapters with explicit type IDs.
- Shared adapter registration and Subscription box name.
- Concrete SubscriptionRepository with persistence-only get, add, update, delete, and clear operations.

Hive IDs and Subscription Field Indices

Type	Hive type ID
------	--------------

UsageFrequency	0
Importance	1
DecisionStatus	2
BillingCycle	3
SavingsAction	4
Subscription	10

Subscription uses explicit @HiveField indices 0 through 14. Nullable fields are DateTime? trialEndDate, double? postTrialPrice,

Tests and Limitation

Phase 2 tests use a temporary filesystem Hive directory and real opened boxes. They cover creation, nullable fields, enum persis

PHASE 3 - DECISION ENGINE

Phase Objective

Implement the single source of truth for Impact Score calculation, KEEP/REVIEW/CUT classification, and NOT ENOUGH INFO handling.

What Was Implemented

- DecisionEngineService in lib/domain/engines/decision_engine_service.dart.
- DecisionEngineResult containing impactScore and decisionStatus.
- Pure evaluation from UsageFrequency and Importance.
- Subscription evaluation that reads only usage and importance.
- Fixed normalization helpers and deterministic threshold classification.

Responsibility and Formula

The service owns calculation and classification. It does not access Hive, prices, trial prices, dates, RevenueCat, notifications

Impact Score = (0.40 x normalized usage) + (0.60 x normalized importance)

Cost is not an input. No price, post-trial price, monthly cost, annual cost, or savings amount is read during scoring.

Usage Normalization

UsageFrequency	Value
daily	1.0
severalPerWeek	0.8
weekly	0.6
monthly	0.4
rarely	0.2
never	0.0

Importance Normalization

Importance	Value
high	1.0
medium	0.5
low	0.0

Classification Thresholds

Impact Score	DecisionStatus
>= 0.70	KEEP
>= 0.40 and < 0.70	REVIEW
< 0.40	CUT

NOT ENOUGH INFO

If usage frequency or importance is missing, the service returns `impactScore = null` and `decisionStatus = notEnoughInfo`. It nev

Tests and Validation

`test/domain/engines/decision_engine_service_test.dart` covers all usage values, all importance values, representative formulas, b

- flutter pub get: passed.
- dart analyze: passed.
- flutter test: passed.
- Focused Decision Engine tests: 11 passed.

Not Implemented

Savings optimizer subset search, candidate selection, overshoot logic, cancellation minimization, tie-breaking, alternatives, Sa

Architectural Decisions and Rationale

- The service is deterministic and side-effect free.
- The scoring formula exists in one place only.
- Cost remains outside Impact Score.
- Missing data is represented explicitly as NOT ENOUGH INFO.
- No arbitrary rounding is applied.
- The immutable Subscription model is not mutated; evaluation returns a result.

Known Limitations

Production startup does not yet initialize Hive or open the Subscription box. Applying engine results to persisted Subscription

How to explain this phase

Trimly now has one small calculator that decides how much a subscription matters. It converts usage and importance into fixed nu

PHASE 4 - OPTIMIZER

What Was Implemented

- Domain-only OptimizerService using exhaustive subset search.
- Separate NaiveBaselineService representing expensive-first cancellation.
- MonthlySavingsCalculator for INR monthly-equivalent savings.
- Immutable optimizer result and explanation models.
- Alternatives from the same exhaustive search, ranked deterministically.
- Trial savings based on postTrialPrice.

Optimizer Architecture

The optimizer consumes existing Subscription values and their impactScore and decisionStatus. It does not access Flutter, Hive,

Candidate Filtering

Only DecisionStatus.review and DecisionStatus.cut subscriptions enter the optimizer search. KEEP, NOT ENOUGH INFO, null-impact,

Monthly-Equivalent Billing Normalization

The MVP supports INR only.

Billing cycle	Monthly-equivalent savings
weekly	price / 0.25
monthly	price
quarterly	price / 3
yearly	price / 12

Normal subscriptions use price. Trial subscriptions use postTrialPrice, then apply the billing-cycle conversion. A trial without

Optimization Algorithm and Ranking

The service evaluates every subset of the eligible candidate pool. A subset is valid only when total monthly savings are greater

Naive Baseline

NaiveBaselineService uses the entire subscription list, including KEEP items, excludes only subscriptions without a valid saving

Alternatives and Explanation

Alternatives are other valid subsets from the same exhaustive search. They use the same ranking order, contain no duplicates, an

Tests and Files

test/domain/engines/optimizer_service_test.dart contains 24 tests covering billing, trials, filtering, targets, ranking, baselin

- flutter pub get: passed.
- dart analyze: passed.
- flutter test: passed.

Phase 4 files: lib/domain/models/optimizer_models.dart, lib/domain/engines/monthly_savings_calculator.dart, lib/domain/engines/o

Architectural Decisions and Limitations

The optimizer remains pure domain logic and does not mutate Subscription records. INR-only support and the existing trialEndDate

CURRENCY & INTERNATIONALIZATION AMENDMENT v1

STATUS: APPROVED
VERSION: v1
STATUS AFTER UPDATE: FROZEN

Currency Model

Trimly supports one selected currency per user. Currency is a user-level setting, not an independently configurable Subscription

Country Selection

Country selection may provide a small local default mapping, such as India to INR, United States to USD, United Kingdom to GBP,

Preserved Rules

Impact Score remains $(0.40 \times \text{normalized usage}) + (0.60 \times \text{normalized importance})$. Currency does not influence Impact Score. Optim

Locale-Aware Display

Future UI must use locale-aware formatting, preferably intl, rather than manual symbol concatenation. User-level settings will c

Approved Non-Changes

This amendment does not implement UserSettings, country or currency pickers, locale formatting, persistence, Subscription schema

PHASE 5 UI/UX SPECIFICATION

STATUS: FROZEN

CURRENCY & INTERNATIONALIZATION

Trimly is globally usable while maintaining one active currency per user. Country selection provides a default currency. Currenc

Currency-Aware Surfaces

Phase 5 must design currency-aware onboarding, Home totals, Savings Mission target/progress, Optimizer target/savings/overshoot/

Explicit Non-Goals

This specification does not authorize UserSettings implementation, country/currency picker UI, intl integration, currency persis

PHASE 5A - USER SETTINGS + CURRENCY FOUNDATION

Objective

Provide the future UI with one authoritative user-level country and currency state, local persistence, deterministic country def

Implemented

- Typed Country and Currency configurations with the approved MVP mapping.
- UserSettings containing only country and currency.
- UserSettingsRepository backed by SharedPreferences.
- Deterministic default settings: India and INR when nothing is saved.
- Riverpod userSettingsProvider and UserSettingsController as the single state source.
- CurrencyFormatter using intl and typed currency locale metadata.

Supported Country Defaults

Country	Currency	Locale
India	INR	en_IN
United States	USD	en_US
United Kingdom	GBP	en_GB
European Union	EUR	de_DE
Japan	JPY	ja_JP
Canada	CAD	en_CA
Australia	AUD	en_AU
Singapore	SGD	en_SG

Persistence and Formatting

Country and currency enum names are stored using concrete SharedPreferences keys. Missing values use India/INR defaults. Formatt

Subscription Compatibility

The existing Phase 2 Subscription.currency field and Hive schema were inspected and left unchanged. UserSettings is the authorit

Tests and Validation

test/features/user_settings_test.dart contains 12 focused tests covering model creation, country defaults, SharedPreferences per

- flutter pub get: passed.
- dart analyze: passed.
- flutter test: passed, including Phase 1-4 tests.

Explicit Non-Goals

No UI, country picker, currency picker, conversion, exchange-rate API, RevenueCat, notifications, milestone logic, Subscription

PHASE 6A - REVENUECAT FOUNDATION

Scope

Phase 6A implements only the RevenueCat entitlement foundation. It does not add payroll UI, purchase UI, offerings, feature gati

Dependency and Configuration

- Added purchases_flutter: ^10.13.1.
- The public SDK key is read from compile-time REVENUECAT_PUBLIC_API_KEY.
- No key is committed. Missing configuration publishes an error state and never grants Pro.

Reactive Architecture

RevenueCat SDK -> RevenueCatService -> CustomerInfo mapping -> premiumProvider -> future UI

RevenueCatConstants.proEntitlementId is pro. The service configures once, retrieves initial CustomerInfo, registers the update l

Entitlement and Error State

The state is loading, free, pro, or error. Pro is granted only when CustomerInfo.entitlements.all['pro']?.isActive == true. Miss

Initialization and Tests

main.dart initializes the singleton service once before starting the existing ProviderScope. No screen initializes RevenueCat an

Explicit Non-Goals

No purchase flow, payroll, offerings, Customer Center, premium feature gating, UI changes, Hive changes, UserSettings changes, D